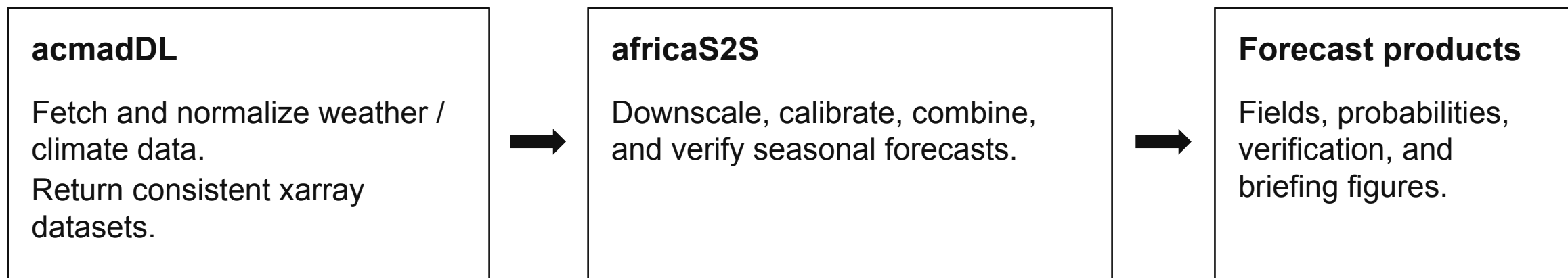


Benchmarking agents for weather forecasting

Which combination of model, libraries, and agent guidance delivers correct forecasts with the least time, tokens, and cost?



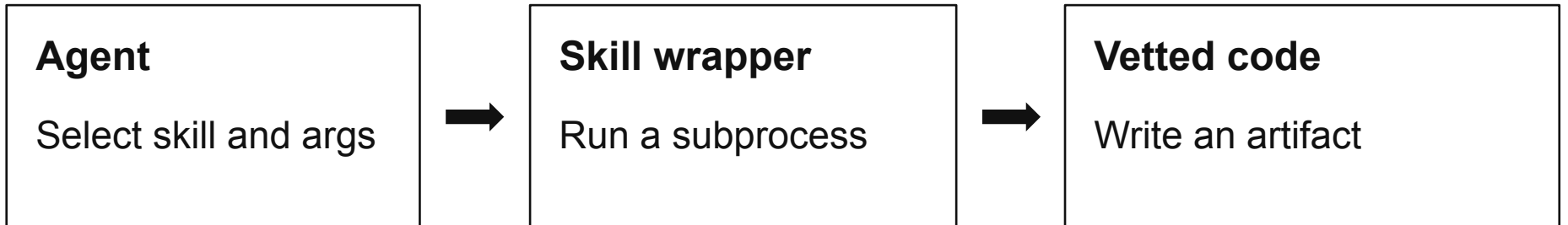
Methods include logistic regression, canonical correlation analysis, quantile mapping, and BCSD.

The benchmark tests the value of reuse against an agent implementing the workflow in Python.

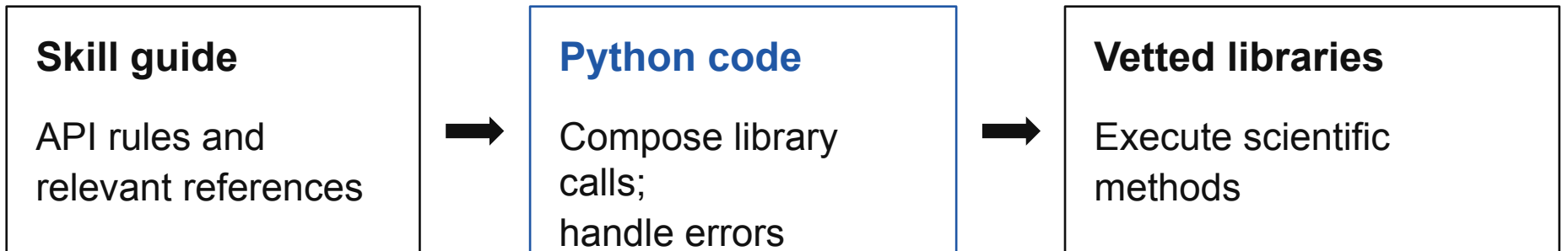
Skills can expose commands or teach library APIs

Both approaches reuse scientific code. They differ in how the agent composes and repairs the workflow.

Command-oriented weather-skills



Library-oriented acmadDL / africaS2S



Hypothesis: progressive disclosure plus Python supports recovery from data, API, and workflow failures.

The current pilot compares three execution conditions

Condition	Write Python	Use weather-skills	Question
Python only	Yes	No	Can the agent implement the task?
Skills only	No	Required	Can catalog operations complete it?
Python + skills	Yes	Available	Does reuse help when code is allowed?

Main comparison: Python only versus Python + skills. Skills only is a diagnostic condition.

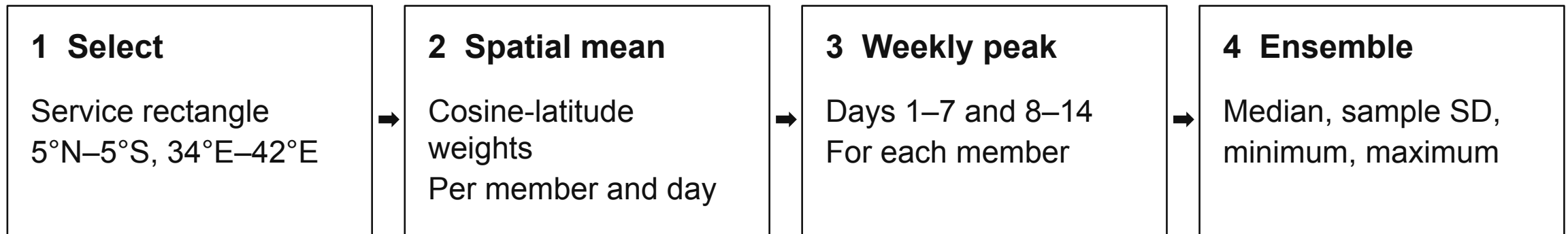
10 models × 3 conditions × 1 task × 1 repetition = 30 planned attempts.

The panel spans frontier models and open-weight models from 1B to 9B.

Astra is excluded because it helped develop the benchmark. Earlier runs remain archived.

Current task: a two-week heat outlook for Kenya

Use a frozen ECMWF S2S forecast to summarize the hottest daily regional-mean 2 m temperature in each week, including ensemble uncertainty.



Required order: spatial mean → weekly peak per member → ensemble statistics.

Deliverables

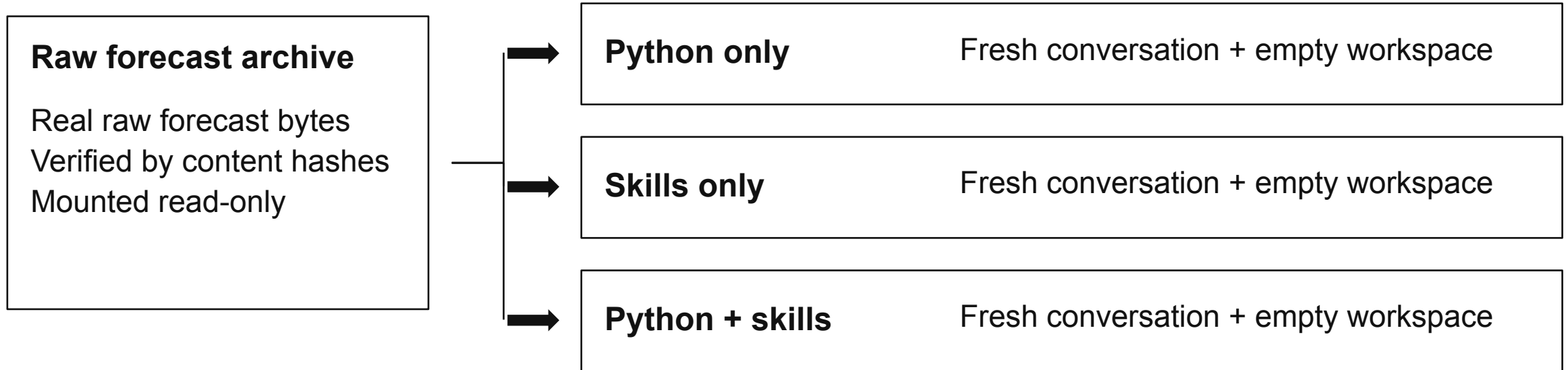
Structured answer, source citation, and a labeled uncertainty figure.

Deterministic checks

Independent numeric reference; exact schema and units; valid, nonblank PNG.

Forecast issue: 2026-09-27. This tests analysis of an issued forecast, not its predictive skill against observations.

Cache the inputs; keep each attempt independent



Within a run

Reuse unchanged successful skill outputs; retain Python artifacts.

Between runs

Share raw inputs only. No generated outputs, intermediate results, or history.

Equal ceilings: 24 model calls · 40 executions · 200k tokens · 600 seconds

Measure correctness, resource use, and failure causes

Task success Numeric reference + output contract

Workflow Guide reads, calls, dependency order

Efficiency Tokens, cost, solve time; cost per success

Diagnostics API errors, invalid actions, execution errors,
budget exhaustion, wrong results

Report service availability separately from task capability.

Where the elapsed time goes

93%

of recorded solve time was spent waiting for model / API responses.

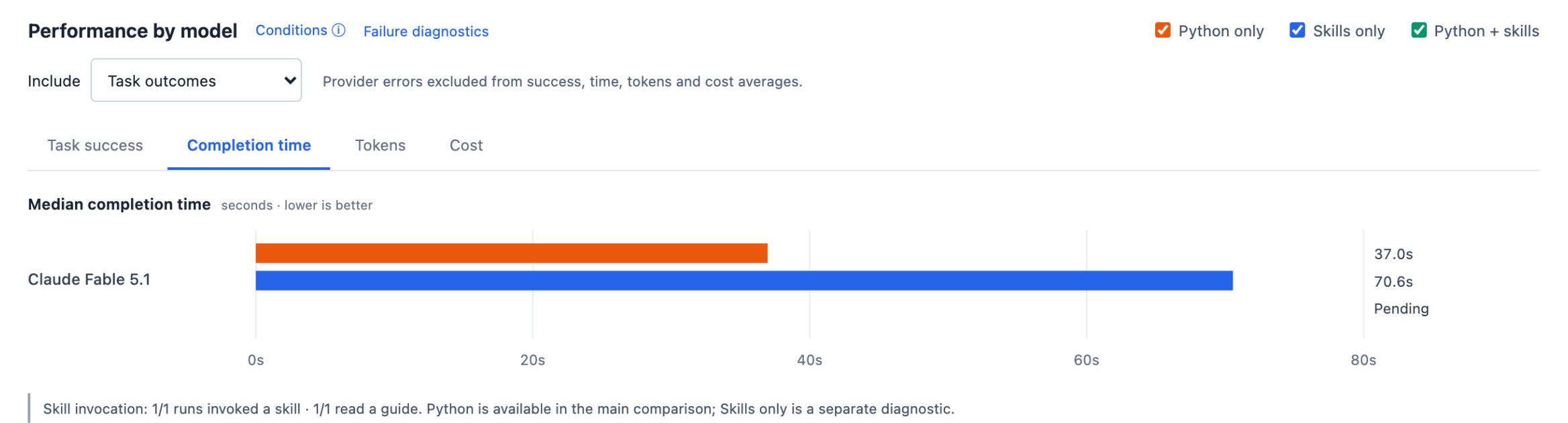


Interim snapshot: 7/30 attempts.

Timing diagnosis, not a model ranking.

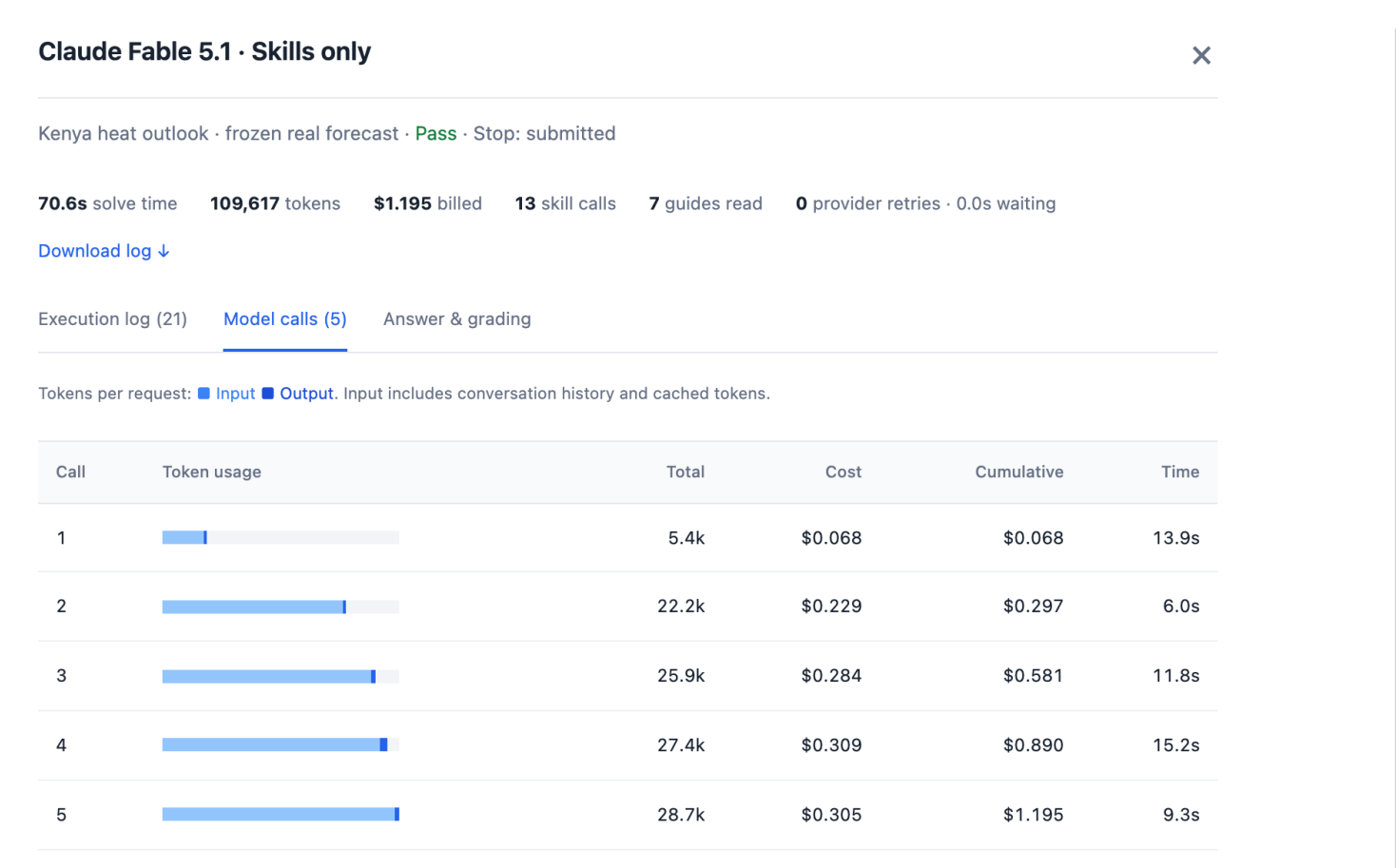
Inspect each comparison in the dashboard

Filter by experiment, model, and task; switch between success, time, tokens, and cost.



Select a bar to inspect its attempts. Screenshot shows one model as an interface example.

Trace a result back to the model and its executions



Each attempt records

Model calls

Visible responses, request time, tokens, and billed cost.

Execution log

Python or skill arguments, outputs, errors, and retries.

Answer & grading

Submitted artifacts, reference values, and failed checks.

Next: isolate the value of the libraries and their skills

The current pilot tests execution patterns. A direct acmadDL / africaS2S study needs its own tasks.

Proposed condition	What the agent receives	What the comparison estimates
Python baseline	General scientific Python	Reference for custom implementations
Python + libraries	Libraries + ordinary API documentation	Value of reusable implementations
Python + libraries + skills	Same libraries + agent skill guides	Added value of progressive disclosure

Seasonal task: acquire hindcasts + observations → align data → fit / downscale → verify on held-out years → produce forecast probabilities and a briefing.

Predeclare tasks and scoring; freeze versions and seeds; repeat across regions, methods, and models.